

Personal Investment OS

External Project Integration Report

Research-only deliverable -- six Taiwan-stock GitHub projects were inspected at the source-code level (not just READMEs) and compared against the existing Personal Investment OS architecture. No application code, schema, migrations, or tests were modified while producing this report.

Repositories inspected:

- openclaw-financial-tw (github.com/cguhenry/openclaw-financial-tw)
- invest-system (github.com/symbiosis11503/invest-system)
- twstock-research (github.com/maplestartend/twstock-research)
- FinMind (github.com/FinMind/FinMind) + OpenAPI spec
- finance-linebot (github.com/KCC6128/finance-linebot)
- kanpan (github.com/benhuang36/kanpan)

1. Executive Summary

- None of the six repos should be adopted wholesale. All are solo/small-team projects (0-15 GitHub stars, most quiet for months); useful as design references, not as dependencies.
- License reality check matters here. openclaw-financial-tw, invest-system, and finance-linebot have no license at all (invest-system's README claims MIT but no LICENSE file exists -- legally that is "all rights reserved"). Their code can be learned from, not copied. Only twstock-research (MIT), kanpan (MIT), and FinMind (Apache-2.0) are safe to directly adapt code from.
- The single highest-leverage finding: FinMind -- our own already-integrated provider -- ships a large catalog of Taiwan-specific datasets (institutional flow, margin/short trading, securities lending, shareholding) that we are not using yet, despite already having paid the integration cost (auth, rate limits, adapter pattern, DTO boundary). This is the cheapest possible expansion.
- The best architecture/design reference is twstock-research -- not because it is popular (2 stars) but because it independently converged on the same layering philosophy we already use (pure domain logic separate from a thin API layer) and did real, carefully-reasoned quant work (regime-aware scoring, Information Coefficient diagnostics with Newey-West correction, look-ahead-bias-safe backtesting). It validates our architecture rather than suggesting we change it.
- None of the six repos does real RAG. Every "AI" feature across all repos is "format structured data into a text prompt, call an LLM once." We do not need a vector database or retrieval infrastructure to reach parity with what these projects call "AI research" -- we need a disciplined context-assembly step, a much smaller lift.
- Recommended next step is NOT an AI chatbot. Per the project's own principle ("deterministic before AI"), the two highest-value, lowest-risk next moves are (a) expand FinMind ingestion to institutional/margin data (cheap, additive, no architecture change) and (b) build a deterministic technical/composite-scoring module in app/analytics/, modeled on twstock-research's design. AI narrative-over-scores is a good Phase 8, not Phase 6.

2. Repository-by-Repository Findings

Repo	Stack	Persistence	Maturity	License	One-line verdict
openclaw-financial-tw	MCP server (27 tools) + FastAPI dashboard + React/TS, lightweight-charts	File-based (JSON/JSONL), no DB	15 commits, 3-week burst, quiet since	None	Best Taiwan-data breadth reference (incl. MOPS scraping), but architecturally thin and legally uncopyable
invest-system	Flask monolith (2,228-line single file) + Backtrader + hand-rolled Canvas K-line	SQLite, no ORM, duplicated schemas	64 commits, 6-week burst, 4 months quiet	None (claimed MIT, no file)	Widest feature scope (70+ routes) but weakest engineering discipline; useful for feature-scope ideas and its Gemini-authored UI redesign spec only
twstock-research	FastAPI + Next.js/React/TS, same domain/API layering as us	SQLite WAL	94 commits, real iteration, 2 months quiet	MIT	Best architecture + quant reference. Directly comparable to our system in philosophy and scale
FinMind	Python package + hosted API	N/A (our provider)	Very active, 2,773 stars	Apache-2.0 (data: non-commercial per README)	Already ours -- but we use ~8 of 75+ datasets and ignore its built-in backtest/indicator modules
finance-linebot	Flask, single LINE webhook	None (in-memory only)	38 commits, 2-day burst, 7 months quiet	None	Thin everywhere except one thing: a genuinely good citation-rebuilding pattern worth studying
kanpan	Tauri (Rust) + React desktop	SQLite (local mirror)	52 commits, AI-assisted, functionally complete	MIT	Best UI-layout reference (chart-dominant detail page, institutional-flow viz); no portfolio/P&L; at all

3. Capability Comparison Matrix

Capability	Ours	openclaw	invest-system	twstock-research	FinMind	finance-linebot	kanpan	Recommendation
Market data (price/OHLCV)	Yes (FinMind)	Yes	Yes	Yes	Yes (source)	Yes	Yes	--
Historical prices	Yes	Yes	Yes	Yes	Yes (30yr)	Latest only	Yes	--
Fundamentals	Yes	Yes	Yes	Yes	Yes	No	Valuation only	--
Valuation (P/E, P/B, yield)	Yes	Yes	Yes	Yes	Yes	No	Yes	--
Dividends	Yes	Yes	Yes	Yes (+event backtest)	Yes	No	Yes	--
Institutional investors (三大法人)	No	Yes	Yes	Yes	Yes (unused by us)	No	Yes	P0 -- add
Margin trading (融資融券)	No	Yes	Yes	Yes	Yes (unused by us)	No	Yes	P0 -- add
Short selling / 融券	No	No	Yes (SBL)	No	Yes (unused)	No	No	P2
Monthly revenue	Schema exists, unpopulated	Yes	Yes	Yes	Yes	No	No	P1
Earnings calls (法說會)	No	No (planned only)	No	No	No	No	No	P3 -- no viable source found
MOPS announcements	No	Yes (scraped)	Partial	Yes (via OpenAPI)	No (gap)	No	No	P1 -- needs TWSE/TPEX direct
News	No	No	Yes (multi-source)	No	Unstructured	Yes	No	P2
Technical indicators	No	Yes (MA/BB/KD/MACD)	Via Backtrader lib	Yes (hand-rolled, TW-correct)	Yes (bundled)	No	Yes	P0 -- add
Factor/composite models	No	ML (Random Forest)	Weighted score	Yes (rigorous, regime-aware)	No	No	No	P1
Stock screening	Yes (basic)	No	Yes	Yes (radar)	No	No	No	P1 -- extend ours
Portfolio analytics	Yes (ours strongest)	No	Paper-trade only	Yes	No	No	No	--
Risk analysis	Basic	No	No	Yes (concentration, ATR stops)	No	No	No	P1
Backtesting	No	No	Yes (Backtrader)	Yes (rigorous, TW-correct)	Yes (bundled!)	No	No	P1 -- Phase 9
Signals	No	Partial	Partial	Yes	Yes (indicators module)	No	Rule text	P1 -- Phase 6/7
Alerts/notifications	No	Yes (Discord/LINE)	Yes (Web Push)	Yes (Discord)	No	No (reply-only)	Yes (desktop native)	P1 -- Phase 7
AI research	No	Partial (external agent)	Yes (debate pattern)	Yes (narrative-over-scores)	No	Yes (grounded reply)	Yes	P2 -- Phase 8
RAG	No	No	No	No	No	Lexical, not vector	No	P3 -- nobody does real RAG; skip
Evidence/citations	No	No	Inline mentions	Implicit via source data	No	Yes (best example)	No	P1 when AI lands
Investment thesis	Yes (ours unique here)	No	No	No	No	No	No	--
Dashboard	Yes	Yes	Yes	Yes	No	N/A	Yes	--
K-line/candlestick UI	No	Yes (lightweight-charts)	Yes (hand-rolled canvas)	Yes (lightweight-charts)	Yes (plotting module)	N/A	Yes (lightweight-charts)	P1 -- add, use lightweight-charts

Capability	Ours	openclaw	invest-system	twstock-research	FinMind	finance-linebot	kanpan	Recommendation
Real-time data	No (by design)	Partial (Fugle)	No	No	No	No	Yes (Fugle WS)	P2 -- deferred
Taiwan ETF support	Partial (Global ETF only)	Partial	No	Partial	Yes (ActiveETF datasets)	No	Badge only	P2
Supply-chain analysis	No	No	No	No	Yes (IndustryChain/MoneyFlow)	No	No	P2

4. Taiwan-Specific Capability Analysis

Mapping the 20 requested Taiwan-specific items against what is implementable without leaving our current provider (FinMind):

Item	FinMind dataset exists?	Any repo implements it well?	Verdict
三大法人 / 外資、投信、自營商	Yes -- TaiwanStockInstitutionalInvestorsBuySell(+Wide)	twstock-research, kanpan	Add now
融資融券	Yes -- TaiwanStockMarginPurchaseShortSale(+Total/Maintenance/Suspension)	twstock-research, kanpan	Add now
借券	Yes -- TaiwanStockSecuritiesLending	invest-system (SBL)	Later -- niche
月營收	Yes -- TaiwanStockMonthRevenue (schema exists, not populated by our adapter)	All	Add now (cheap -- schema ready)
EPS	Yes -- already in our fundamentals	All	Done
法說會	No dataset found anywhere, in any repo	None	Skip -- no viable free source found
除權息	Yes -- already have dividends table (Phase 5B)	twstock-research (event-backtest)	Done for data; event-backtest is a Phase 9 idea
MOPS 重大訊息	No -- not in FinMind	openclaw (scrapes mopsov.twse.com.tw), twstock-research (uses TWSE OpenAPI directly)	Needs a second provider path
公司基本資料	Yes -- TaiwanStockInfo (already used)	All	Done
產業分類	Yes -- industry_category field (already used)	All	Done
供應鏈	Yes -- TaiwanStockIndustryChain(MoneyFlow) (unused)	None implement this	P2 -- untested by anyone, higher risk
台股 ETF	Yes -- TaiwanStockActiveETFInfo/Holding(Change) (unused)	None implement deeply	P2
籌碼面 (broader)	Yes -- TaiwanStockTradingDailyReport (broker branch), TaiwanStockGovernmentBankBuySell, TaiwanStockBlockTrade	invest-system, kanpan	P1 -- real but more niche than institutional/margin
技術面	N/A (compute ourselves)	twstock-research (best), openclaw, kanpan	Add now -- see Section 5
台股交易日	Need our own calendar logic	twstock-research has a proper trading-calendar module	P1 -- needed once date-sensitive features exist
漲跌停	Need our own logic ($\pm 10\%$)	twstock-research models this in backtest fills	P1 -- only matters once we backtest
當沖	No FinMind dataset, no repo implements	None	Skip
融資餘額	Yes -- part of TaiwanStockMarginPurchaseShortSale	twstock-research, kanpan	Bundled with margin-trading add above
法人買賣超	Same as 三大法人 above	--	Bundled above

Bottom line: 6 of the 20 items (institutional flow, margin trading, monthly revenue, broker-branch chip data, industry chain, ETF-specific) are sitting in FinMind's catalog unused, ready to add without a new provider. Only MOPS material announcements genuinely needs a second data path (TWSE/TPEX OpenAPI directly, per the Phase 5 Discovery Report's documented fallback), and earnings calls (法說會) has no viable free source found anywhere across all six repos.

5. Quant / Technical Analysis Recommendations

Metric	In app/analytics/?	Input needed	Data sufficient today?	Deterministic/testable?	Now or later
SMA / EMA	Yes	price_history.close	Yes (once backfilled via FinMind)	Yes, trivially	Now
RSI (Wilder's)	Yes	close series	Yes	Yes	Now
MACD	Yes	close series	Yes	Yes	Now
Bollinger Bands	Yes	close series	Yes	Yes	Now
KD (Taiwan convention: 2/3+1/3 smoothing, NOT textbook Stochastic)	Yes	high/low/close	Yes	Yes	Now -- must use TW convention (twstock-research found generic KD is wrong-signed for TW stocks in their own IC backtest)
Volume ratio / VR (TW 26-day convention)	Yes	volume series	Yes	Yes	Now
Momentum (N-day return)	Yes	close series	Yes	Yes	Now
Volatility (rolling std)	Yes	close series	Yes	Yes	Now
Drawdown (from equity curve)	Yes	portfolio value series	No -- we don't have a portfolio-value time series yet (flagged as deferred in Phase 5 Discovery Report)	Yes, once data exists	Later -- blocked on a Phase-2-engine extension needing approval
Beta	Yes	stock returns + TAIEX returns	Yes (TAIEX index available via FinMind)	Yes	Later -- low priority, nobody in the six repos implements this either
Sharpe / Sortino / Calmar	Yes	trade or equity-curve returns	Only once backtesting exists	Yes	Phase 9 (with backtesting)
ATR (+ Chandelier trailing stop)	Yes	high/low/close	Yes	Yes	P1 -- useful for risk.py-style features even without backtesting
Value/Growth/Momentum/Income style scores	Yes	fundamentals + technical	Yes	Yes	P1 -- straightforward re-weighting of factors we would already compute
Composite score with regime detection	Yes	TAIEX MA200/MA50-slope + all of the above	Yes	Yes	P1 -- most valuable pattern from twstock-research, but do NOT copy their exact weights (tuned on their own IC backtests)
Factor IC diagnostics (Newey-West)	Yes, as a validation tool, not a runtime feature	forward returns + factor scores	Only after enough history accumulates	Yes	Later -- only meaningful once we have months of our own scoring history

Critical correctness lesson from twstock-research, worth internalizing regardless of what we build: their KD scoring is deliberately inverted from textbook logic because they backtested it and found standard "high KD = bullish" has a negative Information Coefficient for Taiwan stocks specifically. This is exactly why "LLM must not calculate these metrics" and "deterministic before AI" matter -- an LLM would confidently apply textbook KD logic and be wrong for this market. Any TA module we build should treat published formulas as a starting point to validate empirically, not gospel.

6. AI / RAG Architecture Recommendations

- Structured data to LLM: universal pattern across finance-linebot, invest-system, kanpan, twstock-research -- none use function-calling/tool-use for data retrieval; all pre-fetch data in plain Python/deterministic code, then serialize into a text block injected into a single prompt. This validates a "stuff already-computed data into context" design over an agentic tool-calling design for V1.
- RAG: none of the six repos implement real RAG (vector embeddings + similarity search). finance-linebot's "RAG" is lexical title/keyword matching over ~16 candidate news articles, not vector retrieval. We do not need a vector DB to match or exceed any of these repos.
- Citations: finance-linebot has the best pattern by far -- it lets the LLM write citation markers [n] inline, then programmatically strips whatever reference section the LLM generated on its own and rebuilds it from the actual source list, so the LLM can never fabricate a source that wasn't retrieved. twstock-research's narrative layer sidesteps the problem differently: it only narrates over already-computed, already-sourced scores (each score's inputs already carry source/as_of metadata from our own schema), so "evidence" is implicit in what was fed in. Both patterns are usable; the second fits our existing source/date convention better.
- Prompt structure: finance-linebot's system prompt explicitly forbids the LLM from being falsely optimistic (不得安抚或模糊回答) and mandates a disclaimer -- directly matches Principle 9 (AI should explain/summarize, not invent) and is worth adapting verbatim in spirit.
- Distinguishing raw data from generated analysis: none of the repos do this cleanly in their data model (it is a prompt-instruction concern, not a schema concern, for all of them). This is a place we can do better than any of the six repos -- our existing investment_thesis table already separates user-authored thesis from system-computed data; an AI-narrative feature should follow the same pattern (a new table with generated_at, model, based_on_snapshot_id -- never mixed into fundamentals/price_history).
- Tool-calling/MCP: FinMind itself now ships an official finmind-mcp package built specifically for Claude Code/Desktop. openclaw-financial-tw's entire architecture is 27 MCP tools wrapping FinMind. This suggests a legitimate alternative Phase 8 design: instead of building a custom "Research Agent" service, expose our own already-computed analytics (holdings, scores, thesis) as an MCP server queried directly in conversation -- no custom LLM-integration code needed in the app at all. Worth flagging as an option, not a decision, since it changes who the "user" of the AI feature is.
- Preventing invented metrics: the only real mechanism across all six repos is "don't let the LLM see raw data it could recompute -- only show it your own already-computed numbers, and instruct it to describe them, not calculate new ones." Our existing Decimal-safe, source-tagged schema is actually a better foundation for this than any of the six repos have, since every fact already carries its provenance.

Proposed "Personal Investment OS to Research Agent" design (describe only, not built)

- Deterministic Analytics (existing + Phase 6 TA/scoring)
- ↓ Research Context Builder (NEW, small service) -- assembles holdings snapshot, thesis, latest scores/signals, each fact tagged with its own source + as_of (reusing existing convention); never includes raw provider JSON, only our own normalized/computed values
- ↓ Prompt Template (narrative-only instructions, explicit "do not calculate" guardrail, mandatory disclaimer, distinguishes FACT / ANALYSIS / UNCERTAINTY per Principle 6)
- ↓ LLM call (Claude, given this is a Claude-Code-authored project -- natural fit)
- ↓ Response stored separately from computed data (new ai_narrative-type table, never mixed into fundamentals/price_history/thesis)

This needs exactly one new service and one new table when it is actually built -- not a new cross-cutting architectural layer. Confirmed by comparing against all six repos: none of them needed more than that either.

7. UI Recommendations

Incremental additions to existing pages only -- no redesign:

- K-line chart on the Research page: 3 of 4 UI-having repos (openclaw, twstock-research, kanpan) independently chose lightweight-charts (TradingView's OSS library) -- a strong convergent signal it is the right choice over Recharts for candlesticks specifically (Recharts stays fine for our allocation pie/line charts). Add as a new chart alongside the existing PriceChart line chart, not a replacement.
- Institutional-flow visualization: kanpan's pattern is worth copying conceptually -- 4 metric cards (外資/投信/自營商/合計, latest day, color-coded) + a 20-day bidirectional bar chart centered on zero (bars up = net buy, down = net sell). Fits naturally as a new Research-page panel once institutional data is added (Sections 4/9).
- Chart-dominant detail page with tabbed secondary data (kanpan): price chart stays always-visible (flex-1), secondary data (fundamentals/valuation/institutional/future-AI) moves into a compact tabbed strip below rather than a long scroll -- worth considering for the Research page once it grows more panels, but this is a layout refactor, not urgent now.
- One canonical stock-summary shape, three renderings (kanpan: grid cards / sortable table / compare chart, same fields): our Watchlist page could adopt a lightweight version of this -- a table/card toggle -- once a live-price column is added (already flagged as an open decision from the Phase 4 report).
- Do NOT adopt: invest-system's hand-rolled Canvas K-line chart (reinvents what a library already solves well, with no tests); any dark-mode-only redesign (our current Tailwind styling is a deliberate "financial terminal, information-dense" choice from the original PRD, not something these repos' visual style should override).

8. Architecture Gap Analysis

Proposed alternative: Provider → Raw Data → Normalization → Evidence → Analytics → Research Context → AI Research Agent, versus current: Provider → DTO → Repository → Service → API → Frontend.

Verdict: the current architecture is sufficient through Phase 6-7 (technical analysis, chip data, alerts). One small, additive change is needed when AI (Phase 8) actually arrives -- not before.

- "Raw Data to Normalization" is already exactly what FinMindProvider does (raw JSON to DTOs) -- no gap.
- "Evidence" is already implicit in our schema (source/as_of/date fields on every market-data row) -- no new layer needed, just discipline in reusing it.
- "Analytics" is already app/analytics/ (P&L;) + the new Phase 6 TA/scoring module -- no gap, just growth.
- "Research Context" is genuinely new -- but it is one service (the Research Context Builder from Section 6), sitting alongside existing services, not a new cross-cutting layer touched by everything else.
- "AI Research Agent" is the consumer of that one new service -- again, a new service, not an architectural rewrite.

One real gap worth naming: MOPS material announcements (Section 4) need a second provider implementation (TWSEProvider / TPEXOpenAPIProvider for that one dataset), since it doesn't exist in FinMind. This was already anticipated in the Phase 5 Discovery Report's "TWSE/TPEX as documented fallback" -- this just gives it a concrete first use case rather than being purely theoretical.

Do not introduce: a generic "evidence" table/entity abstracted across all data types. None of the six projects needed something like this -- every repo that could have used it instead just relied on per-row source columns, exactly like our existing schema already does.

9. Priority Ranking

Answering the seven evaluation questions per item: (1) problem, (2) why not solved already, (3) best reference repo, (4) minimum architectural change, (5) new data required, (6) tests, (7) out of scope -- summarized as structured fact-sheets below, ranked P0 (do next) to P3 (avoid/defer).

1. Institutional investor flow (三大法人) [P0]

Why	Cheapest possible win -- data is already one API call away in our existing provider.
Repo demonstrating	twstock-research, kanpan, openclaw
Data source	FinMind: TaiwanStockInstitutionalInvestorsBuySell
Schema ready?	No -- needs one new table (small, additive)
Provider interface ready?	Yes -- extend MarketDataProvider with one new method, same pattern as get_dividends()
Backend / Frontend / Analytics changes	Small / Small (new Research panel) / None
Complexity / Arch-debt risk	Low / Low

2. Margin / short trading (融資融券) [P0]

Why	Same as above -- high signal value, same data source, same integration cost already paid.
Repo demonstrating	twstock-research, kanpan, invest-system
Data source	FinMind: TaiwanStockMarginPurchaseShortSale
Schema ready?	No -- needs one new table
Provider interface ready?	Yes -- same pattern as above
Backend / Frontend / Analytics changes	Small / Small / None
Complexity / Arch-debt risk	Low / Low

3. Monthly revenue population [P0]

Why	Schema already exists; our FinMind adapter's get_fundamentals() simply doesn't call this dataset yet.
Repo demonstrating	All six
Data source	FinMind: TaiwanStockMonthRevenue
Schema ready?	Mostly (may need one field)
Provider interface ready?	Yes
Backend / Frontend / Analytics changes	Small / Small / None
Complexity / Arch-debt risk	Low / Low

4. Technical indicators (SMA/EMA/RSI/MACD/BB/KD-TW/ATR) [P0]

Why	Foundational for everything else (scoring, signals, alerts); zero AI risk; matches "deterministic before AI".
Repo demonstrating	twstock-research (best), openclaw, kanpan
Data source	Our own price_history
Schema ready?	N/A (computed, not stored -- or cached later)
Provider interface ready?	N/A -- pure app/analytics function
Backend / Frontend / Analytics changes	None (analytics-only) / Small (display on Research) / New module in app/analytics/
Complexity / Arch-debt risk	Medium / Low (isolated, pure functions)

5. K-line chart (lightweight-charts) [P1]

Why	Convergent UI choice across three repos; currently missing entirely.
Repo demonstrating	openclaw, twstock-research, kanpan
Data source	Existing price_history
Schema ready?	N/A
Provider interface ready?	N/A
Backend / Frontend / Analytics changes	None / Medium (new component + dependency) / None
Complexity / Arch-debt risk	Medium / Low

6. Composite / regime-aware scoring [P1]

Why	Highest-value quant idea found, but needs item 4 first and needs our own validation before trusting any weights.
Repo demonstrating	twstock-research
Data source	Our own computed indicators + fundamentals
Schema ready?	Maybe (a scores table)
Provider interface ready?	N/A
Backend / Frontend / Analytics changes	Medium / Medium / New module
Complexity / Arch-debt risk	High / Medium -- weight-tuning is a research project, not a one-time build

7. MOPS material announcements [P1]

Why	Explicitly requested Taiwan feature; needs a second provider.
Repo demonstrating	openclaw (scrape), twstock-research (OpenAPI)
Data source	TWSE/TPEX OpenAPI directly (not FinMind)
Schema ready?	No -- new table
Provider interface ready?	No -- new provider implementation needed
Backend / Frontend / Analytics changes	Medium / Small / None
Complexity / Arch-debt risk	Medium / Low (isolated new provider, matches existing abstraction)

8. Alerts / notifications (Discord) [P1]

Why	Natural Phase 7 per the original roadmap; simplest of all notification patterns seen (webhook, no bot infra).
Repo demonstrating	twstock-research
Data source	N/A (internal)
Schema ready?	No -- new alert_rule table
Provider interface ready?	N/A
Backend / Frontend / Analytics changes	Medium / Medium (Settings UI) / None
Complexity / Arch-debt risk	Medium / Low

9. Broker-branch / chip data (分點) [P2]

Why	Real but more niche; higher scraping/parsing complexity in the repos that hand-roll it.
Repo demonstrating	invest-system, kanpan
Data source	FinMind: TaiwanStockTradingDailyReport
Schema ready?	No -- new table
Provider interface ready?	Yes -- same pattern
Backend / Frontend / Analytics changes	Small / Small / None
Complexity / Arch-debt risk	Low / Low

10. Backtesting engine [P2]

Why	High value but needs items 4/6 first, and look-ahead-bias correctness is nontrivial to get right.
Repo demonstrating	twstock-research (rigorous), invest-system (Backtrader), FinMind itself (bundled)
Data source	Our own historical prices + transactions
Schema ready?	No -- new tables
Provider interface ready?	N/A
Backend / Frontend / Analytics changes	High / Medium / New module, must stay independent of the live calc engine
Complexity / Arch-debt risk	High / Medium -- real risk of subtly-wrong backtests if look-ahead bias isn't handled

11. AI narrative layer [P2]

Why	Valuable but explicitly a later phase, and needs items 4-6 to have anything worth narrating.
Repo demonstrating	twstock-research, finance-linebot (citations)
Data source	Anthropic API
Schema ready?	No -- new table (Section 6)
Provider interface ready?	N/A
Backend / Frontend / Analytics changes	Small (one new service) / Small / None
Complexity / Arch-debt risk	Low if scoped as "narrate only", Medium if scope creeps toward "recommend"

12. Securities lending, industry chain, ETF-specific, supply-chain [P2]

Why	Real Taiwan-specific data, but no repo demonstrates strong value from them and lowest requested priority.
Repo demonstrating	FinMind only (no repo implements well)
Data source	FinMind (various)
Schema ready?	No -- new table each
Provider interface ready?	Yes
Backend / Frontend / Analytics changes	Small each / Small each / None
Complexity / Arch-debt risk	Low each / Low each

13. Full RAG / vector search [P3]

Why	Nobody -- including well-funded-looking attempts -- actually built this; it is not the bottleneck.
Repo demonstrating	None genuinely
Data source	--
Schema ready?	--
Provider interface ready?	--
Backend / Frontend / Analytics changes	--
Complexity / Arch-debt risk	--

14. Real-time streaming (Fugle WS) [P3]

Why	Explicitly deferred in the Phase 5 decision; only kanpan does this, and only because it has no portfolio depth to build instead.
Repo demonstrating	kanpan
Data source	Fugle
Schema ready?	--
Provider interface ready?	Would need a new provider
Backend / Frontend / Analytics changes	--
Complexity / Arch-debt risk	Revisit only when backtesting to paper-trading (Phase 9-10) actually needs live prices

15. AI debate / multi-agent [P3]

Why	Interesting pattern but directly conflicts with the "AI must not independently calculate/conclude important metrics" principle unless heavily constrained; high complexity for unclear value over a single narrative call.
Repo demonstrating	invest-system
Data source	--
Schema ready?	--
Provider interface ready?	--
Backend / Frontend / Analytics changes	--
Complexity / Arch-debt risk	High / High -- the one pattern to actively avoid copying even conceptually without very deliberate guardrails

Ranking your explicit options A-I: D (better screener) and B (Taiwan chip data) and A (technical analysis) are the real next step, roughly in that priority tier (A and B are nearly tied for P0, D follows closely since it is mostly UI/query work over data we are about to have). E (historical performance) is blocked on a Phase-2-engine decision, not this research. F (backtesting) is real but should wait for A/B to mature. C (AI agent) and G (news/RAG) are both later -- C because there is nothing worth narrating yet, G because no repo shows it is worth the complexity. H (better dashboard) is best treated as incremental additions riding on top of A/B/D's new data.

10. Recommended Phase 6 Roadmap

Phase 6 -- Taiwan Chip Data + Technical Analysis (the P0 tier)

- Extend FinMindProvider + MarketDataProvider interface with `get_institutional_flows()`, `get_margin_trading()`; populate monthly revenue via the existing `get_fundamentals()` path.
- New app/analytics/technical.py: SMA/EMA/RSI/MACD/Bollinger/KD(TW convention)/ATR -- pure functions, fully isolated from FastAPI/SQLAlchemy, same independence guarantee as the existing calc engine.
- New schema: institutional_flows, margin_trading tables (same shape/pattern as dividends from Phase 5B).
- Frontend: new Research-page panels for institutional flow (kanpan-style cards + bar chart) and a basic indicator table.
- Extend Screener with the new technical/chip fields.

Phase 7 -- Alerts & Notifications

- Discord webhook (simplest, best-demonstrated pattern -- twstock-research).
- alert_rule table + a rule engine (price/score thresholds), edge-triggered (kanpan's "only fire on unmet-to-met transition" lesson, to avoid spam).
- Settings UI extension (mirrors the Phase 5B "Update Market Data" pattern).

Phase 8 -- Composite Scoring + AI Narrative

- Composite scoring module modeled on twstock-research's design (regime detection, None-aware weighting) -- our own weights, validated against our own data, not copied.
- Research Context Builder service + AI narrative (Claude), strictly narrate-only, stored separately from computed facts.
- MOPS material-announcement provider (second MarketDataProvider implementation, TWSE/TPEX direct).

Phase 9 -- Backtesting

- Only after Phase 6-8 give the engine something real to test. Must implement look-ahead-bias protection (publish-date-aware queries) and TW-specific limit-up/down fill logic from day one, per twstock-research's demonstrated lessons.

Phase 10 -- Paper Trading / Real-time (revisit)

- Only if Phase 9's backtesting validates a strategy worth watching live. Fugle's real-time tier becomes relevant here, not before.

11. Deferred Features

Real-time market data (Fugle), full RAG/vector search, AI debate/multi-agent systems, 當沖 (day-trading) metrics, 法說會 (earnings call) tracking (no viable free source found), supply-chain/industry-chain analysis, broker-branch chip data, securities lending, ETF-specific tooling. None of these are wrong to want eventually -- they are just lower-value or higher-risk than the P0/P1 tier given what these six repos actually demonstrate.

12. Risks

- Composite scoring weight-tuning is an ongoing research effort, not a one-time build -- twstock-research's own commit history shows continuous refinement based on IC backtests. Budget for this as recurring work.
- MOPS scraping fragility: both openclaw and invest-system that scrape MOPS directly (vs. via an OpenAPI) show this is more fragile than API-based sources -- prefer TWSE/TPEX's own OpenAPI announcement endpoints over scraping mopsov.twse.com.tw where possible.
- AI scope creep toward "debate"/recommendation patterns (invest-system) directly conflicts with the human-in-the-loop principle -- worth an explicit reminder before Phase 8 starts.
- FinMind's non-commercial data clause (already documented in our README) applies with equal force to every new dataset added from its catalog -- no new legal exposure, but no reduction either.

13. License / Data-Use Considerations

Provider	Code license	Can copy code?	Data redistribution	Commercial use	Notes
law-financial-tw	None	No	N/A	N/A	All rights reserved by default
invest-system	None (claimed MIT, no file)	No (unenforceable claim, not a grant)	N/A	N/A	Legally ambiguous -- treat as unlicensed
twstock-research	MIT	Yes, with attribution	Data separately documented per-source in DATA_SOURCES.md	Code: yes. Data: depends on source	Best-documented licensing hygiene of the six
openclaw	Apache-2.0 (code)	Yes	Data: "educational, non-commercial" per README	Data use restricted, code use not	Already our situation -- no change
twlinebot	None	No	N/A	N/A	--
finmind	MIT	Yes, with attribution	N/A (no bundled data)	Yes	--
twse/tpeex OpenAPI	Government Open Data License v1	N/A (data, not code)	Yes, with attribution	Yes	Already our documented fallback (Phase 5)
twstock / yfinance	N/A	N/A	No -- explicitly forbidden by Yahoo ToS	No	twstock-research never commits Yahoo data to their repo for this reason -- good practice to mirror

The core distinction to not confuse: MIT/Apache-2.0 licenses on these repos cover their code. None of them grants any right to redistribute the underlying financial data -- that is governed separately by each data source's own terms (TWSE/TPEX: open, FinMind: non-commercial, Yahoo: forbidden). We already handle this correctly for FinMind (documented in our README); the same discipline applies to anything new added from its catalog.

14. Exact Implementation Recommendations

For every "yes, do this" item, answering the seven-question framework explicitly:

Institutional flow + margin trading (P0)

1. Problem	No 籌碼面 data exists in our system at all -- a core Taiwan-investor need.
2. Why not solved already?	Phase 5B only implemented price/fundamentals/dividends/valuation methods on MarketDataProvider; institutional/margin were out of scope, not rejected.
3. Best reference	FinMind's own dataset (TaiwanStockInstitutionalInvestorsBuySell, TaiwanStockMarginPurchaseShortSale) -- not any repo's code (all three that use it re-fetch the same FinMind endpoint we already have access to; nothing to copy, just new adapter methods, same pattern as get_dividends()/get_valuation() from Phase 5B).
4. Minimum architectural change	Two new MarketDataProvider abstract methods + two new DTOs + two new tables + two new repository classes -- mechanically identical to how get_dividends() was added. Zero changes to app/analytics/.
5. New data required	None beyond what FinMind already provides at our current tier.
6. Tests	Mirror Phase 5B's test suite shape exactly -- DTO normalization (mocked HTTP), provider contract, ingestion upsert/partial-failure/rate-limit tests, migration correctness.
7. Out of scope	Any interpretation of the flow data (e.g. "3 days of net foreign buying = bullish") -- that is Phase 8 scoring/AI territory, not this step.

Technical indicators module (P0)

1. Problem	No technical analysis exists; Screener and Research pages have no momentum/trend view.
2. Why not solved already?	Never built -- Phases 1-5 focused on portfolio accounting and data ingestion, not analysis.
3. Best reference	twstock-research/app/indicators/technical.py (MIT -- safe to closely study formula-by-formula, especially their TW-corrected KD convention and documented rationale comments). Do not literally copy files (different schema/DB access patterns) -- reimplement the formulas as pure functions against our own PriceHistory-shaped input.
4. Minimum architectural change	New app/analytics/technical.py module. Zero changes to existing calc-engine files, zero new tables if we compute on-demand rather than caching (recommended for V1).
5. New data required	None -- uses price_history we already have.
6. Tests	Pure-function unit tests with hand-computed expected values for each indicator (same style as existing app/analytics tests) -- the easiest-to-test item in this whole report.
7. Out of scope	Composite scoring, regime detection, ML -- those come after this foundational module exists and only once indicator correctness is validated against known values.

MOPS material announcements (P1)

1. Problem	Explicitly requested Taiwan feature (重大訊息), no path today.
2. Why not solved already?	FinMind doesn't have this dataset at all (confirmed by direct inspection) -- a genuine provider gap, not an oversight.
3. Best reference	twstock-research uses TWSE/TPEX's own OpenAPI announcement endpoints (cleaner than openclaw's HTML scraping of mopsov.twse.com.tw) -- prefer that approach.
4. Minimum architectural change	A second concrete MarketDataProvider implementation (e.g. TWSEAnnouncementProvider) used only for this one capability -- the first real use of the multi-provider design already built for in Phase 3/5.
5. New data required	TWSE/TPEX OpenAPI direct calls (already documented as our approved fallback provider).
6. Tests	Same DTO-normalization + contract-test pattern as FinMindProvider.
7. Out of scope	Any AI summarization of announcement content -- surface the raw announcement list first.

K-line chart (P1)

1. Problem	Research page has a line chart, not a candlestick chart -- weaker for pattern-reading than every comparable repo offers.
2. Why not solved already?	Recharts (our current charting library) has no good native candlestick component; never prioritized.
3. Best reference	kanpan's PriceChart.tsx (MIT) -- direct, safe reference for lightweight-charts API usage (candlestick series, volume histogram, MA overlay lines, crosshair legend).
4. Minimum architectural change	New frontend dependency (lightweight-charts) + one new React component, consuming the exact same /api/prices/{ticker} data already served. Zero backend changes.
5. New data required	None.
6. Tests	A component-level test confirming it renders given mock price data (matching existing frontend test patterns).
7. Out of scope	Real-time updates (no live data yet, per the Phase 5 decision), drawing tools, multi-timeframe intraday (only daily closes exist).

Composite scoring (P1, do only after the above)

1. Problem	No way to compare/rank stocks beyond the existing basic screener.
2. Why not solved already?	Needs technical indicators (above) as a prerequisite; never built.
3. Best reference	twstock-research/app/scoring/ (MIT) for design pattern (None-aware weighted mean, regime-aware weight tables, style sub-scores) -- NOT their specific weights or thresholds, which were tuned against their own IC backtests with no evidence they apply to our universe.
4. Minimum architectural change	New app/analytics/scoring.py (pure functions, same independence rule) + a decision on whether scores are computed on-demand or cached in a new table (recommend: start on-demand, matching indicators above).
5. New data required	TAIEX index history (for regime detection) -- one more FinMind dataset (TaiwanStockPrice on the index, or TaiwanStockTotalReturnIndex), not a new provider.
6. Tests	Unit tests per sub-score with hand-computed cases, plus explicit "missing data returns None, never a fake neutral value" tests (twstock-research's most important design lesson).
7. Out of scope	Any live IC/Newey-West statistical validation -- that requires months of our own scoring history to be meaningful and is a Phase 9+ concern, not part of building the scorer itself.

No code was written or modified while producing this report. Next step: choose which P0 item (institutional flow, margin trading, monthly revenue, or technical indicators) to start with, or reorder the Phase 6 scope before implementation begins.